Document number: P3919R0
Audience: EWG

Ville Voutilainen
2025-11-05

Guaranteed-(quick-)enforced contracts

Abstract

This paper explains the needs (or if you dare to agree to call them that, requirements) that we have for guaranteed-enforced contract assertions.

The overall need

Multiple users in WG21 and outside it report a need to have contract assertions that are always enforced, either with the 'enforce' semantic or the 'quick_enforce' semantic.

Such contract assertions are a memory-safety facility. That is why they need to be always-enforced, so that the subsequent code that they protect is guaranteed not to be entered if the predicate of the assertion evaluates to false.

This is similar to the requirements of a hardened standard library. It's paramount that when hardened, violations of hardened preconditions never ever lead to the library function to be entered, possibly hitting undefined behavior in the function.

The requirements here are fundamentally different from the ones of P2900 contracts. The next section goes into more detail on that.

The differences to P2900

A guaranteed assertion must be enforced exactly once

A guaranteed assertion must be enforced at least once, but that turns out to be exactly once. The reason for this is that since these assertions are always-enforced, that needs to be as low-overhead as possible, to avoid suggesting to users that there's a performance reason *not* to use them. Granted, while deployment experience with such assertions suggests that such performance reasons do not actually exist, a significant reason for that is that the deployed solutions do not enforce the assertions any more times than exactly once.

A guaranteed function contract assertion must be enforced even if the call to a function is indirect. Both for free functions and member functions, so calls via pointers to functions and via pointers to member functions also have to enforce the assertions. Otherwise the desired guarantee is lost, and the function contract assertion doesn't provide the memory-safety it has to provide.

That means that *semantically*, the assertions have to be enforced as-if they are enforced in the function definition.

That does *not* mean that any of the suggested implementation strategies in P3267 are precluded; any as-if implementation is allowed, *as always*. But this semantic requirement needs to be there nonetheless, there

cannot be room for allowing the contracts to not be enforced because a caller-client-side enforcement strategy is chosen and that then can't enforce in case an indirection is used, because the caller-client site wouldn't see a function contract assertion due to the use of an indirection.

A guaranteed assertion must not translate exceptions to contract violations

The exception translation that P2900 contract assertions do has a cost. There might be work-arounds that alleviate that cost, but many work-arounds don't completely get rid of the (code size and run-time performance) cost of the translation.

It's of fundamental importance that memory-safety assertions are as low-cost as possible - again to avoid suggesting to users not to use them, and because they are always "on", always enforced.

Furthermore, always-enforced assertions are not "ghost code". There's no actual surprise if they let exceptions escape, and that never happens in a conditional fashion depending on any build mode.

Both of those aspects make the P2900 for exception translation not hold, but the efficiency one is compelling in and of itself.

The solution to these aspects is to make it so that if the predicate throws an exception, the contract assertion doesn't handle that exception in any way, i.e. lets it through, i.e. propagates it, i.e. throws the same exception.

Constification is harmful for guaranteed assertions

For a memory-safety facility, it's rather useful to understand what the predicate expression is, and not require that understanding to understand an invisible translation of the expression.

There are also non-quantified costs for all sorts of analyses of such expressions for various tools, including optimizers, and also including static analysis tools. The translated expressions mean that there's more expressions to analyze, more expressions to prove equivalent, and costs thereof. While the design of P2900 may have chosen not to really investigate those costs, the prudent approach is to not have those costs at all for memory-safety assertions.

The nature of being always-enforced also removes a large part of the concerns about side effects. When always-enforced assertions are enforced exactly once, side effects are manageable. While it might sound like a nice idea to discourage side effects, constification indeed merely discourages them, but doesn't prevent them. There's no concerns about how many times side effects occur, and no concerns about whether they occur. They occur. Once.

Other interactions with P2900

A relatively obvious question is what should happen if P2900 contracts and guaranteed-enforced contracts are mixed in one function declaration, i.e.

```
void f(int* p) pre!(p) pre(*p != 42);
```

The easy part is the guaranteed enforcing of the guaranteed-enforced contract - including the part where it's guaranteed to be enforced even when the function is called indirectly. Those requirements are paramount, and non-negotiable.

What we should do here, then, is that all contract assertions, whether always-enforced or flexible-semantic ones, are evaluated in the lexical order. Therefore the `pre!(p)` above should be evaluated and enforced before the `pre(*p != 42)` is

What follows from that is that we can't have the different contracts be evaluated on different sides of a caller/definition boundary, that is, we can't evaluate the flexible-semantic contracts on the caller-client-side and the enforced ones on the definition-side, because that could mean that the order changes. So for such mixtures, we need to evaluate all function contract assertions of a function on the definition-side (to make sure the guaranteed-enforced ones are enforced even when calling indirectly through a pointer-to-function).

Future considerations, and virtual functions

Guaranteed-enforced contracts must be enforced, and they must be enforced even if indirections are used for the call.

Which means that we have to guarantee such evaluations on the definition-side, and that will mean that guaranteed-enforced contracts are automatically inherited, so that by default they are there, enabled, and enforced, and that doesn't need to be separately requested.

Whether a derived class is allowed to explicitly turn off a base class contract is a separate consideration that can certainly be entertained. There are arguments both ways, and this separate concern needs to be looked at and prototyped, but entertaining such an ability to turn off a guaranteed-enforced base class contract can be done after contracts on virtual functions are added to C++, because it's plausibly going to be an explicit opt-in with additional syntax, not a different meaning for a plainer syntax.